

AWS ও Server Tools

সিনিয়র লেভেল রিভিশন — ১৫ মিনিট। ফোকাস: একটা Laravel/React প্রোডাক্ট AWS-এ প্রোডাকশনে চালানোর বাস্তব জ্ঞান — সার্টিফিকেশন-থিওরি নয়।

১. একটা Laravel অ্যাপের স্ট্যান্ডার্ড AWS Deployment Architecture

Description: "আপনার অ্যাপ AWS-এ কীভাবে deploy করবেন?" — সবচেয়ে সম্ভাব্য ওপেনিং প্রশ্ন। একটা পরিষ্কার reference architecture মুখস্থ থাকলে বাকি প্রশ্ন এর ডালপালা।

৬টা Key Point:

1. Core স্ট্যাক: Route 53 (DNS) → CloudFront (static/CDN) → ALB → EC2 Auto Scaling Group / ECS (app) → RDS (MySQL) + ElastiCache (Redis) + S3 (files)
2. App server stateless রাখা — session/cache Redis-এ, uploads S3-তে; তবেই horizontal scale আর instance মরলেও ক্ষতি নেই
3. Queue worker আলাদা instance/service-এ — web আর worker-এর scale-এর কারণ ভিন্ন; scheduler একটাতেই (বা EventBridge)
4. Network ভাগ: public subnet-এ শুধু ALB/NAT; app আর DB private subnet-এ — DB কখনোই public নয়
5. React frontend: S3 + CloudFront static hosting — EC2-তে serve করার দরকারই নেই; দাম প্রায় শূন্য
6. Multi-AZ সবখানে (ALB, ASG, RDS standby) — এক AZ পড়লেও সার্ভিস চলে; BD-স্কেল প্রোডাক্টে multi-region সাধারণত overkill

২. EC2 vs ECS/Fargate vs Lambda — Compute বাছাই

Description: কোন workload কোন compute-এ চালাবেন — cost, control, operational overhead-এর trade-off বলা।

৬টা Key Point:

1. EC2: পূর্ণ control, সম্ভাব্য reserved-এ — কিন্তু patching/AMI/scaling নিজের ঘাড়ে; traditional সেটআপ
2. ECS + Fargate: container দাও, সার্ভার ভুলে যাও — patching নেই, per-task scale; আজকের দিনে Laravel-এর জন্য sweet spot
3. Lambda: event-driven ছোট কাজ (image resize, webhook, scheduled task) — ১৫ মিনিট limit, cold start; পুরো Laravel অ্যাপ চালানো (Vapor বাদে) সাধারণ পছন্দ নয়
4. সিদ্ধান্ত-নিয়ম: steady traffic → EC2/ECS reserved; spiky/অনিয়মিত → Fargate/Lambda pay-per-use
5. ECS টার্ম জানা: task definition (container spec), service (desired count + LB যুক্ত), cluster — deploy মানে নতুন task definition revision-এ rolling update
6. যেটাই হোক, image/AMI immutable — সার্ভারে ঢুকে hotfix করা anti-pattern; বদল মানে নতুন deploy

৩. RDS: Production Database অপারেশন

Description: Managed DB নিলে কী পান, কী তবু নিজের দায়িত্ব — backup, replica, failover, connection সামলানো।

৬টা Key Point:

1. RDS দেয়: automated backup + point-in-time recovery, patching, Multi-AZ failover — undifferentiated কাজ AWS-এর ঘাড়ে

2. Multi-AZ (sync standby, failover ~১-২ মিনিট) ≠ read replica (async, read scale) — দুটোর উদ্দেশ্য আলাদা, দুটোই লাগতে পারে
3. Failover-এ endpoint DNS একই থাকে — app-এ retry logic থাকলে প্রায় স্বচ্ছভাবে পার হয়
4. Connection সীমা মাথায় রাখা: অনেক app server/Lambda হলে RDS Proxy দিয়ে pooling — নাহলে **too many connections**
5. তবু নিজের দায়িত্ব: query optimization, index, slow query log দেখা — managed মানে DBA-র কাজ উধাও নয়
6. Snapshot restore drill করা + খরচ নজরে: instance right-sizing, gp3 storage, dev/staging রাতে বন্ধ — BD context-এ cost প্রশ্ন প্রায় নিশ্চিত

8. S3 ও CloudFront: File Storage ও CDN

Description: User upload, report file, static asset — S3-কেন্দ্রিক ডিজাইন আর নিরাপদ file access-এর প্যাটার্ন।

৬টা Key Point:

1. S3 = ১১টা 9 durability-র object store — app server-এর ডিস্কে file রাখা মানেই scale ও reliability দুটোই ভাঙা
2. Private bucket + pre-signed URL — file access-এর সঠিক প্যাটার্ন: সীমিত সময়ের signed link, bucket কখনো public নয়
3. Upload-ও direct-to-S3 (pre-signed PUT) করা যায় — বড় ফাইল app server-এর ভেতর দিয়ে টানা অপচয়
4. Lifecycle policy: পুরনো report/log → Infrequent Access → Glacier → delete — storage বিল নিয়ন্ত্রণের প্রধান অস্ত্র
5. CloudFront: edge cache — static asset আর React bundle-এর latency কমায়ে; BD ইউজারের জন্য নিকটবর্তী edge থেকে serve
6. Block Public Access সবসময় on + bucket policy দিয়ে শুধু CloudFront (OAC) access — S3 data leak নিউজে আসা সবচেয়ে কমন ভুল

৫. IAM: Security-র মেরুদণ্ড

Description: Access key লিক হওয়াই AWS-এর এক নম্বর নিরাপত্তা দুর্ঘটনা। Role-ভিত্তিক least-privilege ডিজাইন বলতে পারা সিনিয়র লক্ষণ।

৬টা Key Point:

1. মূলনীতি least privilege: যতটুকু দরকার ততটুকু permission — **AdministratorAccess** app-কে দেওয়া অপরাধ
2. EC2/ECS-এ IAM Role ব্যবহার — কোডে/env-এ access key রাখা নয়; role-এর temporary credential auto-rotate হয়
3. User (মানুষ, MFA বাধ্যতামূলক) vs Role (সার্ভিস, assume করা হয়) — পার্থক্যটা পরিষ্কার বলা
4. Policy structure: Effect + Action + Resource + Condition — resource-লেভেলে সীমিত করা (**arn:aws:s3:::my-bucket/reports/***)
5. Secret management: DB password/API key → Secrets Manager বা SSM Parameter Store — git-এ .env কখনো নয়
6. Audit: CloudTrail-এ কে কী করেছে সব লগ + billing alert — অস্বাভাবিক activity (মাইনিং instance) ধরার শেষ রক্ষা

৬. Auto Scaling ও Load Balancing

Description: ট্রাফিক বাড়াকমার সাথে ইনফ্রা কীভাবে নিজে adjust করবে — health check, scaling policy, আর graceful deploy।

৬টা Key Point:

1. ALB: L7 (HTTP) — path/host routing, health check fail হলে instance-এ traffic বন্ধ; NLB শুধু raw TCP/চরম performance দরকারে
2. Health check endpoint সচেতনভাবে বানানো (/health: DB+Redis ping) — শুধু 200 ফেরালে অর্ধমৃত instance-ও healthy দেখায়
3. Scaling policy: target tracking (CPU ৬০%-এ রাখা) সবচেয়ে সহজ ও যথেষ্ট; scheduled scaling — জানা পিক (মাসের শেষে রিপোর্ট সিজন) আগেভাগে
4. Scale-in protection/connection draining — instance নামানোর আগে চলমান request শেষ করতে দেওয়া
5. Worker-এর scaling metric CPU নয়, queue depth — CloudWatch custom metric দিয়ে
6. Warm-up সময় হিসাবে রাখা: নতুন instance আসতে কয়েক মিনিট — খুব spiky হলে headroom/faster boot (container) দরকার

৭. Docker ও Container Workflow

Description: Dev থেকে প্রোডাকশন পর্যন্ত একই environment — image build, layer optimization, আর compose vs orchestration-এর সীমা।

৬টা Key Point:

1. মূল লাভ: "আমার মেশিনে চলে" সমস্যার মৃত্যু — image-ই artifact, সব environment-এ একই জিনিস চলে
2. Multi-stage build: composer/npm build stage আলাদা, final image-এ শুধু runtime — ছোট image, কম attack surface
3. Layer caching: কম বদলানো জিনিস (dependency install) আগে, কোড copy পরে — build সময় কয়েক গুণ কমে
4. Container ephemeral: ভেতরে state/upload রাখা নয় — volume বা S3; log stdout-এ (docker logs/CloudWatch নেয়)
5. docker-compose local dev-এর জন্য (app + MySQL + Redis এক কমান্ডে); প্রোডাকশন orchestration → ECS/K8s — compose প্রোডাকশন টুল নয়
6. Image hygiene: specific tag (latest নয়), non-root user, `.dockerignore`, ECR-এ vulnerability scan

৮. CI/CD Pipeline ডিজাইন

Description: Push থেকে প্রোডাকশন পর্যন্ত automated পথ — কী কী stage, কোথায় মানুষ লাগে, আর রোলব্যাক কীভাবে।

৬টা Key Point:

1. স্ট্যান্ডার্ড pipeline: push → lint + test → build (Docker image/artifact) → staging deploy → (approval) → production deploy
2. Laravel deploy ধাপগুলো স্ক্রিপ্টে: composer install `--no-dev`, config/route cache, `migrate --force`, queue worker restart (`queue:restart`)
3. Zero-downtime: rolling deploy (ECS নতুন task আগে, পুরনো পরে নামে) বা symlink switch (Deployer-স্টাইল) — কোনোটা request drop নয়
4. Migration backward-compatible রাখা deploy-এর শর্ত — পুরনো কোড চলাকালীন নতুন schema-ও কাজ করবে (দুই-ধাপ column drop)
5. Rollback plan আগে থেকে: আগের image/release-এ ফেরা এক কমান্ডে — DB rollback প্রায় অসম্ভব বলেই migration সাবধানে
6. Secret pipeline-এ inject (GitHub Actions secrets/SSM) — repo-তে কখনো নয়; প্রতিটা deploy trace-able (কে, কোন commit, কখন)

৯. Monitoring, Logging ও Alerting

Description: "প্রোডাকশনে সমস্যা হলে কীভাবে জানবেন?" — observability সেটআপ না থাকলে বাকি architecture-এর দাম নেই।

৬টা Key Point:

1. তিন স্তর: metrics (CloudWatch), logs (centralized — CloudWatch Logs/ELK), error tracking (Sentry) — তিনটাই লাগে, একটার বদলে আরেকটা নয়
2. Alert actionable ও অল্প: 5xx rate, p95 latency, queue depth, DB CPU, disk — শতটা noisy alert মানে কেউ দেখে না
3. Log structured (JSON) + correlation/request ID — এক request-এর পথ web → queue → job পর্যন্ত trace করা যায়
4. App-level মেট্রিকও দরকার: order/minute হঠাৎ শূন্য — সার্ভার সবুজ কিন্তু business মরে থাকা ধরা পড়ে
5. Sentry-জাতীয় টুলে exception group + release track — নতুন deploy-এর পর error spike সরাসরি দেখা যায়
6. Uptime bahar থেকেও চেক (external ping) — নিজের infra-র ভেতরের monitor নিজের সাথেই ডুবতে পারে

১০. Linux Server অপারেশন Essentials

Description: Managed যুগেও সিনিয়রকে সার্ভারে ঢুকে সমস্যা খুঁজতে জানতে হয় — "সাইট slow/down, SSH করলেন, তারপর কী?"

৬টা Key Point:

1. দ্রুত triage রুটিন: `top/htop` (CPU/mem কে খাচ্ছে) → `df -h` (disk full? — সবচেয়ে কমন কারণ) → `free -m` → সার্ভিস status
2. Log দেখা: `journalctl -u nginx -f`, `tail -f app log`, nginx error log — error message-ই ৮০% ক্ষেত্রে উত্তর বলে দেয়
3. Port/process: `ss -tlnp` কোন port-এ কে শুনছে; `ps aux | grep` — "502 Bad Gateway" মানে প্রায়ই php-fpm/app process মরে আছে
4. systemd দিয়ে সার্ভিস: unit file-এ `Restart=always` — queue worker/daemon crash করলে নিজে ওঠে; Supervisor-ও একই কাজের প্রচলিত টুল
5. Nginx ভূমিকা বলতে পারা: reverse proxy + static serve + TLS termination — php-fpm/Node upstream-এ; বেসিক config (root, location, proxy_pass) চেনা
6. নিরাপত্তা বেসিক: SSH key-only (password off), ufw/security group-এ শুধু দরকারি port, fail2ban, নিয়মিত patch — public EC2-র ন্যূনতম স্বাস্থ্যবিধি